ANHANGUERA

ENGENHARIA DE SOFTWARE

GABRIEL DAS CHAGAS SILVA

Análise e visualização de vendas

Linguagem de Programação

Unidade 3 - Introdução à análise de dados | Aula 4 - Visualização de dados

2026

GABRIEL DAS CHAGAS SILVA

Análise e visualização de vendas

Relatório de aula prática apresentado à Anhanguera, no curso de Engenharia de Software, para a disciplina Linguagem de Programação.

Aluno: Gabriel das Chagas Silva
Disciplina: Linguagem de Programação
Unidade 3 - Introdução à análise de dados | Aula 4 - Visualização de dados
Data: 13 de outubro de 2026

Ambiente de execução

Os códigos e testes desta entrega foram executados localmente em Python. Os notebooks .ipynb incluídos são compatíveis com o Google Colab e contêm as saídas reais da execução. As capturas mostram o notebook executado localmente; não representam uma sessão no serviço Colab.

Preparação e resultados

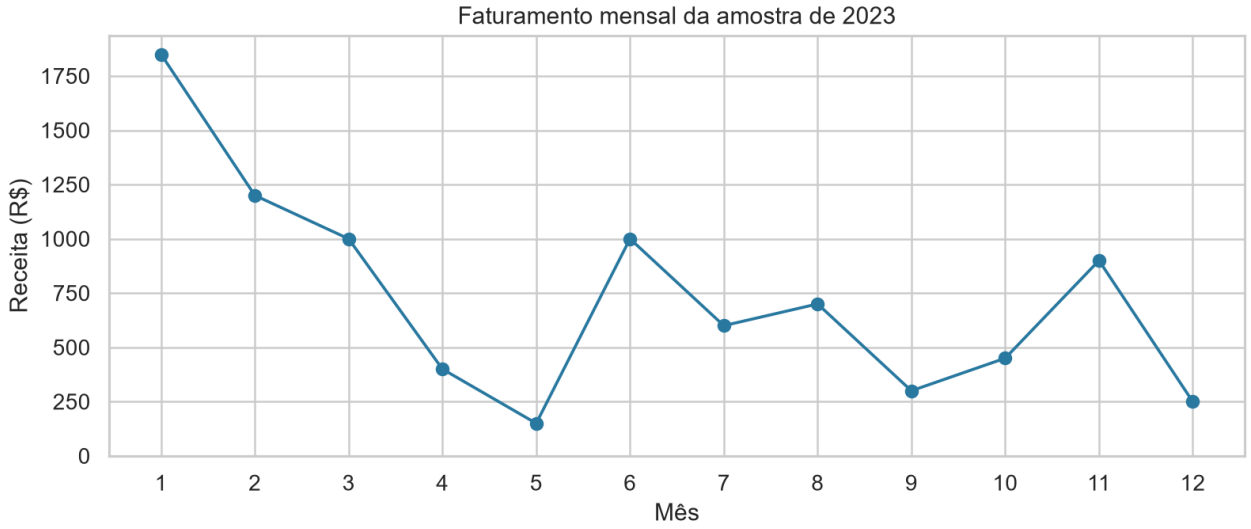
O banco SQLite dados_vendas.db contém a tabela vendas1 com os 14 registros de 2023 fornecidos no roteiro. A tabela demonstrativa é reiniciada antes de inserir a amostra, evitando duplicação ao executar novamente. Pandas lê a consulta SQL em um DataFrame, converte datas e valores e verifica ausências, duplicatas e valores negativos. A amostra não apresentou esses problemas.

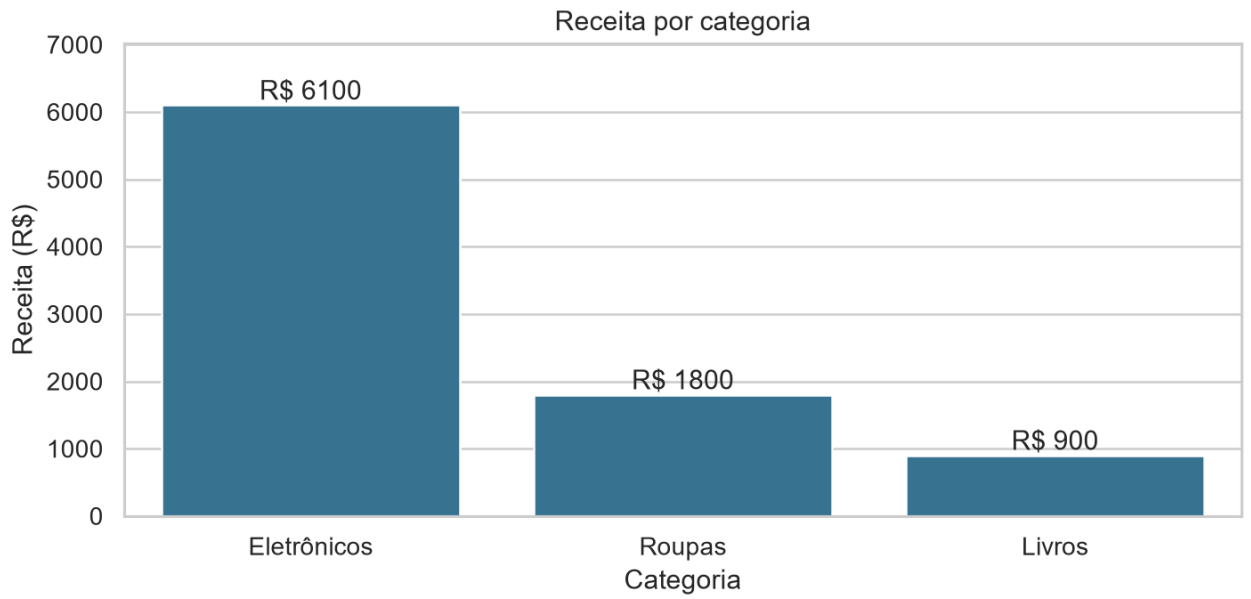
Indicador	Resultado
Receita total	R$ 8.800,00
Registros de venda	14
Ticket médio	R$ 628,57
Maior receita mensal	Janeiro: R$ 1.850,00
Menor receita mensal	Maio: R$ 150,00

Categoria	Receita	Participação
Eletrônicos	R$ 6.100,00	69,32%
Roupas	R$ 1.800,00	20,45%
Livros	R$ 900,00	10,23%

Ticket médio é a média do valor dos registros de venda disponíveis. Receita não equivale a lucro: não há custos ou margens no conjunto de dados.

Visualizações





Insights e avaliação dos resultados

Eletrônicos responde pela maior parcela da receita da amostra. É razoável investigar disponibilidade e demanda nessa categoria e testar ações comerciais acompanhadas de margem e retorno. Janeiro apresentou a maior receita registrada; maio, a menor. Antes de atribuir isso a sazonalidade, é preciso ampliar a base: 14 vendas são insuficientes para concluir um padrão anual da empresa.

Recomenda-se coletar mais transações, quantidades vendidas, custos, estoque e campanhas. Com esses dados, podem ser comparadas receitas, margens e resultados de promoções. Os gráficos ajudam a formular hipóteses, mas não comprovam as causas das diferenças observadas.

Como Pandas, Matplotlib e Seaborn apoiam decisões no varejo?

SQLite organiza os registros e permite consultas. Pandas prepara, filtra e agrega os dados, produzindo indicadores consistentes. Matplotlib controla a apresentação dos gráficos, enquanto Seaborn facilita visualizações estatísticas e comparações. Juntos, permitem identificar categorias relevantes, acompanhar evolução e comunicar resultados para orientar hipóteses sobre estoque, campanhas e mix de produtos. A qualidade da decisão depende da representatividade dos dados e da verificação dessas hipóteses.

Os testes conferiram 14 registros, receita total de R$ 8.800,00, somas das categorias, ausência de valores negativos e identificação dos meses máximo e mínimo. Os resultados correspondem aos dados de exemplo de 2023, e não ao último ano civil atual.

Referências

Roteiro U3 A4, material fornecido. Documento fornecido para a atividade, s.d.

Pandas, read_sql_query. Disponível em: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_sql_query.html. Acesso em: 13 out. 2026.

Seaborn, introdução. Disponível em: https://seaborn.pydata.org/introduction.html. Acesso em: 13 out. 2026.

Codificação completa

Arquivo vendas.py. Inclui os comentários, as verificações e a gravação dos resultados.

```
# %% Conectar ao SQLite e inserir a amostra fornecida pelo roteiro
import sqlite3
import json
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

dados = [
    ('2023-01-01', 'Produto A', 'Eletrônicos', 1500.00),
    ('2023-01-05', 'Produto B', 'Roupas', 350.00),
    ('2023-02-10', 'Produto C', 'Eletrônicos', 1200.00),
    ('2023-03-15', 'Produto D', 'Livros', 200.00),
    ('2023-03-20', 'Produto E', 'Eletrônicos', 800.00),
    ('2023-04-02', 'Produto F', 'Roupas', 400.00),
    ('2023-05-05', 'Produto G', 'Livros', 150.00),
    ('2023-06-10', 'Produto H', 'Eletrônicos', 1000.00),
    ('2023-07-20', 'Produto I', 'Roupas', 600.00),
    ('2023-08-25', 'Produto J', 'Eletrônicos', 700.00),
    ('2023-09-30', 'Produto K', 'Livros', 300.00),
    ('2023-10-05', 'Produto L', 'Roupas', 450.00),
    ('2023-11-15', 'Produto M', 'Eletrônicos', 900.00),
    ('2023-12-20', 'Produto N', 'Livros', 250.00),
]
with sqlite3.connect('dados_vendas.db') as conexao:
    conexao.execute('''CREATE TABLE IF NOT EXISTS vendas1 (
        id_venda INTEGER PRIMARY KEY AUTOINCREMENT,
        data_venda DATE, produto TEXT, categoria TEXT, valor_venda REAL)''')
    # Reinicia somente a tabela demonstrativa para evitar duplicar os exemplos.
    conexao.execute('DELETE FROM vendas1')
    conexao.executemany('''INSERT INTO vendas1
        (data_venda, produto, categoria, valor_venda) VALUES (?, ?, ?, ?)''', dados)
    conexao.commit()
    df_vendas = pd.read_sql_query('SELECT * FROM vendas1', conexao)

# %% Exploração e preparação dos dados
print('AMOSTRA DOS DADOS')
print(df_vendas.head().to_string(index=False))
print('\nValores ausentes:', df_vendas.isna().sum().to_dict())
df_vendas['data_venda'] = pd.to_datetime(df_vendas['data_venda'], errors='raise')
df_vendas['valor_venda'] = pd.to_numeric(df_vendas['valor_venda'], errors='raise')
assert df_vendas.notna().all().all()
assert (df_vendas['valor_venda'] >= 0).all()
assert not df_vendas.duplicated(
    ['data_venda', 'produto', 'categoria', 'valor_venda']).any()
print('Duplicatas: 0; dados preparados:', df_vendas.dtypes.to_dict())

# %% Análises de faturamento e categorias
mensal = df_vendas.groupby(df_vendas['data_venda'].dt.month)['valor_venda'].sum()
mensal = mensal.reindex(range(1, 13), fill_value=0)
categorias = df_vendas.groupby('categoria')['valor_venda'].sum().sort_values(ascending=False)
total = float(df_vendas['valor_venda'].sum())
ticket = float(df_vendas['valor_venda'].mean())
participacao = categorias / total * 100
print(f'Faturamento: R$ {total:.2f}; vendas: {len(df_vendas)}; ticket: R$ {ticket:.2f}')
```

Codificação completa continuação

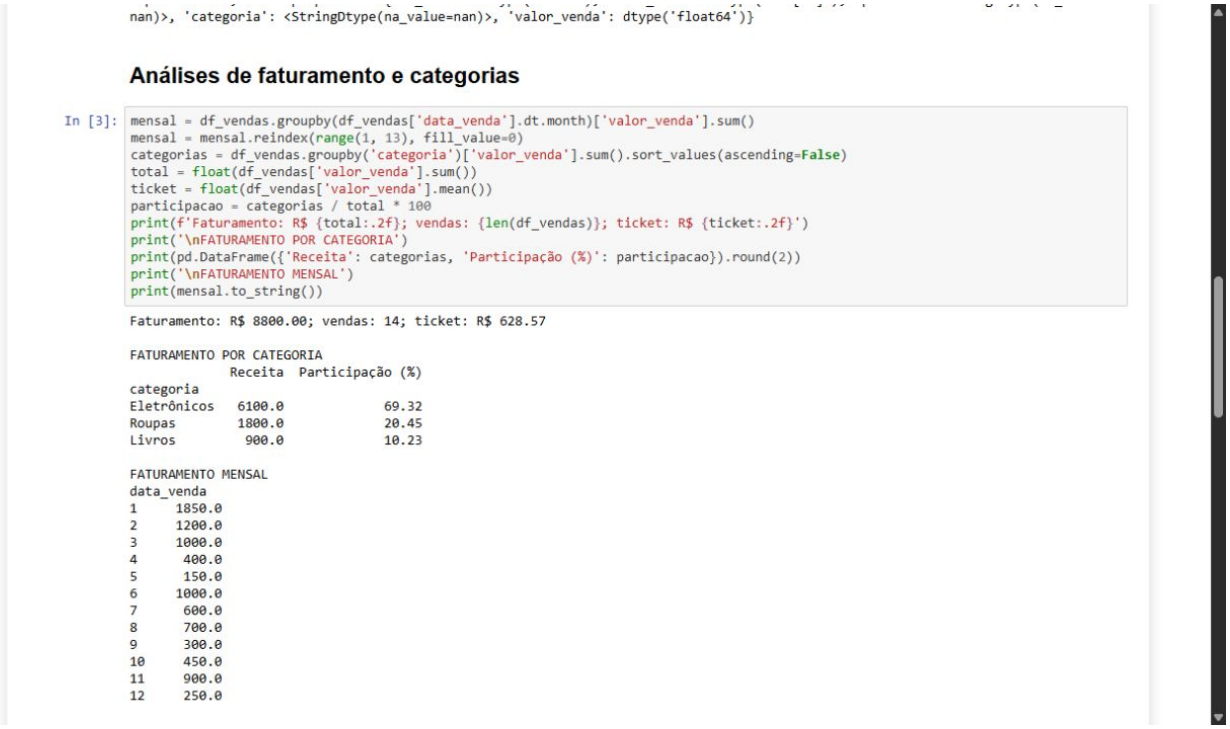
```
print('\nFATURAMENTO POR CATEGORIA')
print(pd.DataFrame({'Receita': categorias, 'Participação (%)': participacao}).round(2))
print('\nFATURAMENTO MENSAL')
print(mensal.to_string())

# %% Visualizações com Matplotlib e Seaborn
sns.set_theme(style='whitegrid')
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(mensal.index, mensal.values, marker='o', color='#2878a0')
ax.set(title='Faturamento mensal da amostra de 2023', xlabel='Mês', ylabel='Receita (R$)')
ax.set_xticks(range(1, 13))
ax.set_ylim(bottom=0)
fig.tight_layout()
fig.savefig('vendas_mensais.png', dpi=180)
plt.show()
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(x=categorias.index, y=categorias.values, color='#2878a0', ax=ax)
ax.set(title='Receita por categoria', xlabel='Categoria', ylabel='Receita (R$)')
for barra in ax.patches:
    ax.annotate(f'R$ {barra.get_height():.0f}',
                (barra.get_x() + barra.get_width()/2, barra.get_height()),
                ha='center', va='bottom')
ax.set_ylim(0, categorias.max() * 1.15)
fig.tight_layout()
fig.savefig('vendas_categorias.png', dpi=180)
plt.show()

# %% Conferência e registro dos resultados
assert len(df_vendas) == 14 and round(total, 2) == 8800.00
assert categorias.to_dict() == {'Eletrônicos': 6100.0, 'Roupas': 1800.0, 'Livros': 900.0}
assert mensal.idxmax() == 1 and mensal.idxmin() == 5
print('TESTES: 14 registros, valores válidos e totais conferidos.')
with open('resultados.json', 'w', encoding='utf-8') as arquivo:
    json.dump({'total': total, 'vendas': 14, 'ticket': ticket,
               'categorias': categorias.to_dict(), 'mensal': mensal.to_dict(),
               'participacao': participacao.to_dict()}, arquivo, ensure_ascii=False, indent=2)
```

Registro de execução

Captura do notebook com código e saída efetivamente executados no ambiente local. As demais células e saídas estão preservadas no arquivo .ipynb.



nan)>, 'categoria': <StringDtype(na_value=nan)>, 'valor_venda': dtype('float64')}

Análises de faturamento e categorias

In [3]:
```
mensal = df_vendas.groupby(df_vendas['data_venda'].dt.month)['valor_venda'].sum()
mensal = mensal.reindex(range(1, 13), fill_value=0)
categorias = df_vendas.groupby('categoria')['valor_venda'].sum().sort_values(ascending=False)
total = float(df_vendas['valor_venda'].sum())
ticket = float(df_vendas['valor_venda'].mean())
participacao = categorias / total * 100
print(f'Faturamento: R$ {total:.2f}; vendas: {len(df_vendas)}; ticket: R$ {ticket:.2f}')
print('\nFATURAMENTO POR CATEGORIA')
print(pd.DataFrame({'Receita': categorias, 'Participação (%)': participacao}).round(2))
print('\nFATURAMENTO MENSAL')
print(mensal.to_string())
```

```
Faturamento: R$ 8800.00; vendas: 14; ticket: R$ 628.57

FATURAMENTO POR CATEGORIA
            Receita  Participação (%)
categoria
Eletrônicos   6100.0             69.32
Roupas        1800.0             20.45
Livros         900.0             10.23

FATURAMENTO MENSAL
data_venda
1     1850.0
2     1200.0
3     1000.0
4      400.0
5      150.0
6     1000.0
7      600.0
8      700.0
9      300.0
10     450.0
11     900.0
12     250.0
```